OmicMiningAgent — Data Foundation *(typically first)*

Planner: *"ALWAYS run FIRST to ground analysis in actual expression data — establishes data foundation."*

Note: ordering shown is a *typical* realisation; the LLM planner may reorder, skip, or loop back to any agent based on intermediate results.

▼ Reads from AgentState

- `query` (str) — user's natural-language research question
- `session_id` (str) — unique session identifier for file paths and result caching

System Prompt Highlights (from `OmicMiningAgent.system_message`)

1. Extract `disease`, `organ`, `cell_type` from the user query
2. Map free-text to canonical forms: "Alzheimer's" → `"Alzheimer disease"`, "Lung cancer" → `"lung adenocarcinoma"`
3. Map organs: Alzheimer's → `"brain"`, Breast cancer → `"breast"`
4. After tool calls → "write a concise summary of the gene list and pathway data"

Tool: `omic_analysis_tool(disease, organ, cell_type)`

Pipeline: GLiNER NER → OmniCellTOSG scRNA-seq query → DESeq2 differential expression → clusterProfiler KEGG/GO enrichment

Returns: sample counts (disease vs. normal), DEG rankings with log2FC & FDR, volcano plots, KEGG/GO enrichment dotplots

Resources: OmniCellTOSG · GLiNER · DESeq2/R · clusterProfiler · BiomedBERT

▲ Writes to AgentState (via `_extract_structured_data`)

- `top_genes[]` (list[str]) — ranked DEGs with log2FC & FDR e.g. `[MAP3K15, CFAP47, IL3RA, ...]`
- `disease` (str) — canonical disease name e.g. `"Alzheimer disease"`
- `cell_type` (str) — cell-type filter if specified e.g. `"microglial cell"`
- `pathways[]` (list) — KEGG and GO enrichment results
- `deg_results` (dict) — full differential-expression statistics per gene
- `plot_paths` (dict) — volcano plot, KEGG dotplot, enrichment plot file paths

↓ *Consumed by:* ***BioMarkerKGAgent*** *(`top_genes`)* · ***PubMedResearcher*** *(`top_genes`)* · ***ScientistsAgent*** *(`top_genes`, `pathways`)* · ***ReportingNode*** *(`plot_paths`)*

Figure 1: **OmicMiningAgent** — workflow interface view. The ordering shown here reflects a *typical* plan generated by the LLM orchestrator; in practice the planner dynamically decides which agents to invoke, in what order, and whether to loop back after failures or surprising results. That said, the planning prompt strongly encourages running this agent first to "ground ALL hypotheses in actual omics data." It reads only the user query and session identifier, maps them to canonical disease/organ/cell-type forms via its system prompt, and invokes a single tool (`omic_analysis_tool`) that chains NER, database retrieval, differential expression, and pathway enrichment. The six fields it writes to `AgentState`—most critically `top_genes[]`—become the shared anchor that every downstream agent reads.

BioMarkerKGAgent — Knowledge-Graph Contextualisation

Planner: *"Query top 10–15 DEGs for KG neighbours — builds Gene→Pathway→Phenotype chains."*

Shown as a second step in this example; the planner may invoke it earlier, later, or re-invoke it after replanning.

▼ Reads from AgentState

- `query` (str) — original research question
- `top_genes` (list[str]) — ranked DEGs *from OmicMiningAgent*
- `previous_results` (str) — accumulated text output from Step 1

System Prompt Highlights (from `BioMarkerKGAgent.system_message`)

1. "Query the PrimeKG knowledge graph to retrieve relevant biomedical information"
2. "Provide structured data and relationships from the knowledge graph to enhance understanding"
3. *(From planning prompt)* Query for *both* up-regulated and down-regulated gene sets
4. *(From planning prompt)* Be THOROUGH — query KG for multiple gene sets to build Gene→Pathway→Phenotype chains

Tool: `biomarker_kg_tool(query)`

Internal pipeline: text embedding → PrimeKG subgraph retrieval (PCST) → GAT graph encoder → Llama-3.1-8B decoder → ranked hypotheses

Returns: drug interactions, pathway memberships, GO terms, protein–protein interactors, disease associations

Resources: Neo4j/PrimeKG (20k+ diseases, 30k+ proteins) · GRetriever micro-service (port 8001) · OpenAI embeddings · PyTorch Geometric

See companion illustration for internal GRetriever architecture.

▲ Writes to AgentState

- `agent_outputs[]` (str) — structured KG results per gene, including:
  - — pathway membership chains (e.g. *CDK5* → *cell-cycle signalling* → *neuronal death*)
  - — GO biological-process terms
  - — drug–gene interactions and existing therapeutic targets
  - — protein–protein interaction partners

↓ *Consumed by:* ***PubMedResearcher*** *(pathway context)* · ***ScientistsAgent*** *(KG evidence for hypothesis anchoring)* · ***ReportingNode*** *(KG analysis section)*

Figure 2: **BioMarkerKGAgent** — workflow interface view. In the typical realisation shown here this agent runs second, but the LLM planner can reorder, skip, or re-invoke any agent at any point—for instance, the planner may call it again after literature search reveals new gene targets. When invoked, it reads the `top_genes` list from `AgentState` and queries the PrimeKG knowledge graph for each gene's neighbourhood, targeting the top 10–15 DEGs to build explicit Gene→Pathway→Phenotype chains. Internally, the tool communicates with a GRetriever micro-service that extracts PCST subgraphs, encodes them with a Graph Attention Network, and decodes answers with Llama-3.1-8B (see companion illustration for architecture details). Its output—appended to `agent_outputs[]`—provides the pathway and interaction context that downstream agents use to anchor hypotheses in structured biomedical knowledge.

PubMedResearcher — Literature Validation

Planner: *"Literature on TOP targets AND pathways — validates & classifies novelty."*

Shown as a third step here; the planner may invoke it earlier (e.g. for a literature-first query) or re-invoke it after replanning.

▼ Reads from AgentState

- `top_genes` (list[str]) — ranked DEGs *from OmicMiningAgent*; prompt says "search for ALL of them"
- `query` (str) — original research question
- `previous_results` (str) — accumulated output from Steps 1–2 (omics + KG context)

System Prompt Highlights (from `PubMedResearcher.system_message`)

1. "Check `context["top_genes"]` for genes from previous analysis — search for ALL of them"
2. "Group related genes into 2–4 focused queries for efficiency" (e.g. `"TREM2 TYROBP microglia Alzheimer"`)
3. "`pubmed_full_search_tool` is your PRIMARY tool — it downloads and reads full papers"
4. "Every claim MUST include a citation: (Author et al., Year, DOI/PMID)"
5. Output per gene: **Function**, **Disease Role**, **Key Findings**, **Therapeutic Potential**
6. "Synthesise findings ACROSS papers — identify convergent and contradictory evidence"

Tool 1 (primary): `pubmed_full_search_tool(query, num_papers=50)`

Downloads full-text PDFs/XMLs from PubMed Central, Elsevier, Wiley, arXiv, bioRxiv, medRxiv via paperscraper. Max 100 papers per call. Papers cached globally for speed. Returns metadata + abstract + extracted full text.

Tool 2 (fallback): `pubmed_lite_tool(query)`

`[DEPRECATED]` — lightweight NCBI E-utilities search. Abstracts and metadata only, no PDF download. $\leq$20 papers. "Use only when speed is critical."

See companion illustration for source landscape (PubMed, PMC, Elsevier, Wiley, arXiv, bioRxiv, medRxiv).

▲ Writes to AgentState (via regex extraction from LLM output)

- `paper_dois[]` (list[str]) — extracted DOI strings for traceability
- `paper_pmids[]` (list[int]) — extracted PubMed ID numbers
- `agent_outputs[]` (str) — per-gene literature reviews structured as:
  - — **Function**: biological role with citations
  - — **Disease Role**: mechanism linking gene to condition
  - — **Key Findings**: significant results across papers
  - — **Therapeutic Potential**: drug targets or interventions
  - — **References**: all papers cited with DOI/PMID

↓ *Consumed by:* ***ScientistsAgent*** *(`paper_dois`, literature context)* · ***ReportingNode*** *(citations, literature-validated targets section)*

Figure 3: **PubMedResearcher** — workflow interface view. Although shown here in the third position, the planner may invoke this agent at any point—even first for purely literature-driven queries—and may re-invoke it after replanning if earlier agents fail or surface unexpected targets. It provides the evidence layer. Its system prompt instructs it to read *all* genes from `top_genes` and group them into 2–4 focused queries for efficient retrieval. The primary tool downloads up to 100 full-text papers per call with global caching; a deprecated lite fallback is available for speed-critical lookups (see companion illustration for source landscape). Every claim is required to carry a citation. The structured per-gene output and extracted DOIs/PMIDs it writes to `AgentState` provide the literature grounding that ScientistsAgent uses for hypothesis novelty classification and that ReportingNode uses for the citation apparatus.

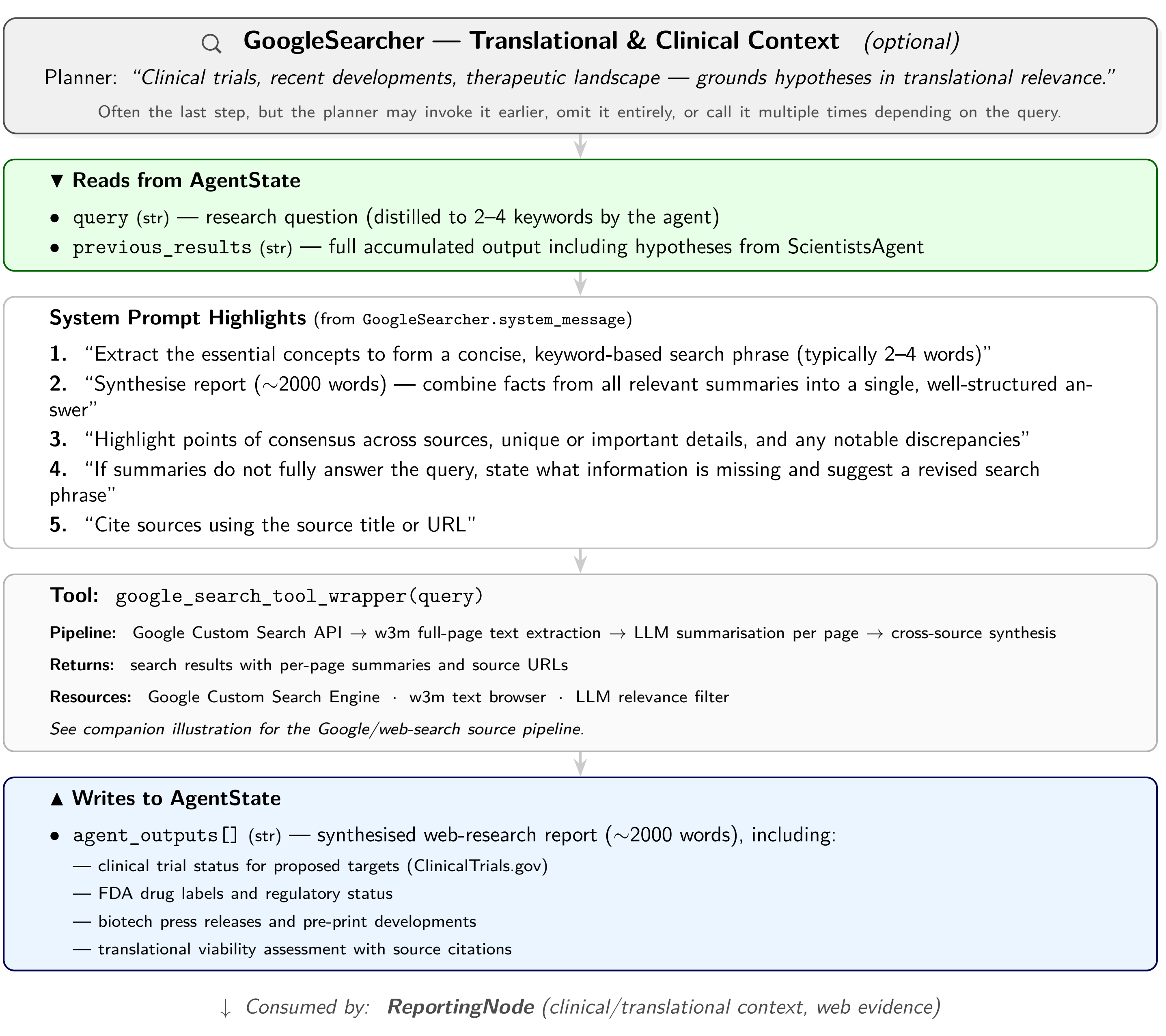

Figure 4: **GoogleSearcher** — workflow interface view. This agent is typically invoked late in the pipeline—often after ScientistsAgent has produced mechanistic hypotheses—but the planner may invoke it at any point, omit it for purely molecular queries, or re-invoke it after a replan cycle. Its system prompt defines a disciplined three-step workflow: (1) distil the query into 2–4 core keywords, (2) execute a targeted Google Custom Search, and (3) synthesise the retrieved page summaries into a cohesive ∼2000-word report. Unlike the PubMed agent, its value lies in surfacing information *not* indexed in biomedical databases—clinical trial registries, FDA labels, biotech press releases, and pre-prints—thereby grounding the proposed targets in translational reality (see companion illustration for the web-search pipeline).

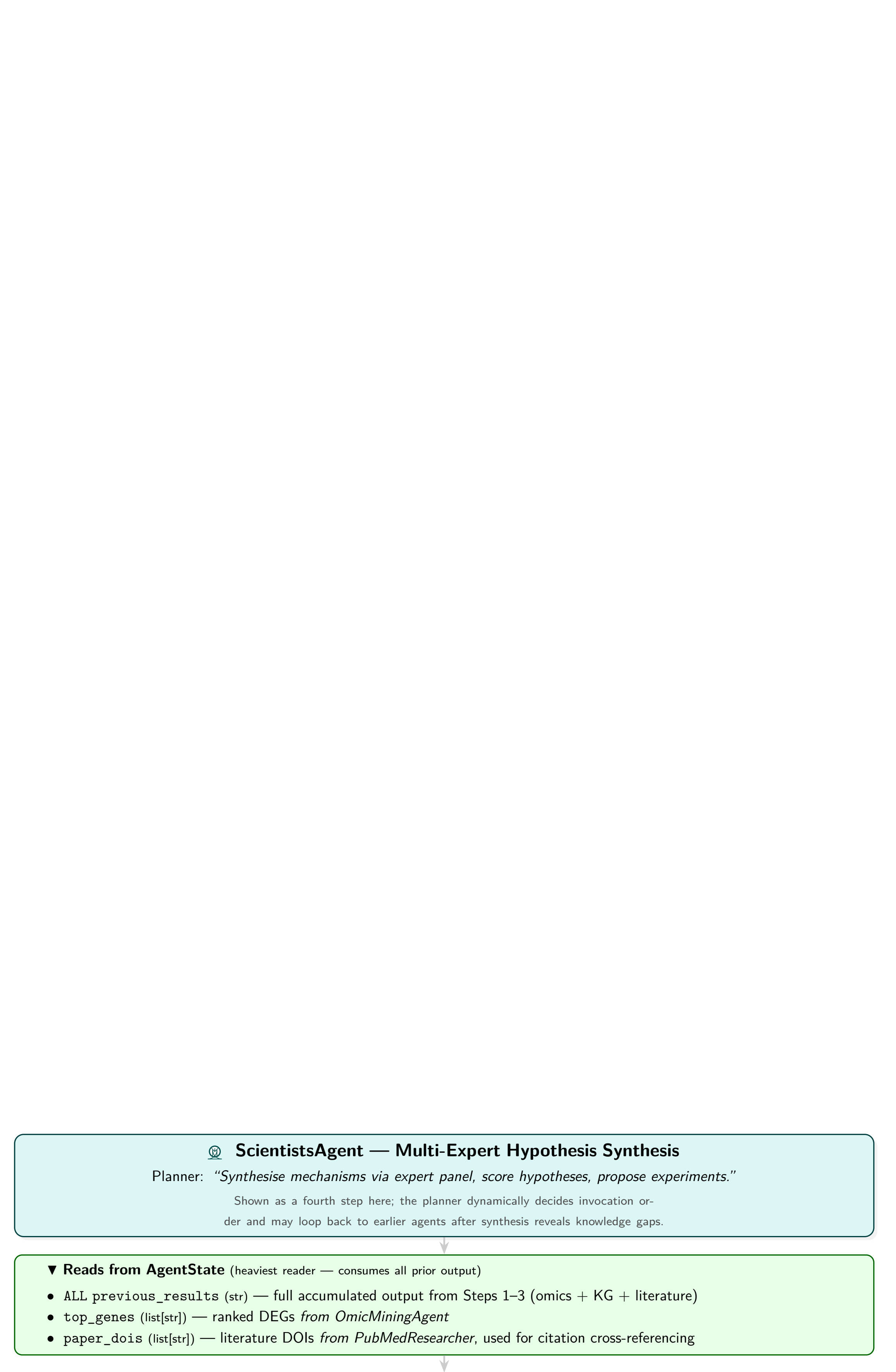

Figure 5: **ScientistsAgent** — workflow interface view. The heaviest reader of `AgentState`, this compound agent consumes all prior omics, KG, and literature output. A PI-level router uses a structured `ExpertRouting` Pydantic model to select 2–3 of the four domain experts configured in **`configs/experts.yaml`** and dispatch tailored sub-queries. Genomics, Neuroscience, and Longevity/Biostatistics experts are backed by curated-author HyperRAG KBs (*RAG KB*); CancerExpert is configured but its KB is not yet built and is therefore routed *LLM-only*. The PI synthesis node integrates expert perspectives into ranked hypotheses with proposed validation experiments.

Figure 6: **Author Knowledge-Base Build Pipeline.** A gitignored **`experts.yaml`** maps each expert alias to a private scientist name used *only* for PubMed queries. The offline build (Stages 1–6) fetches papers, extracts text, chunks, embeds, and runs LLM entity/relation extraction. At query-time the pre-built KB loads from cache in seconds. This separation provides (i) *modularity* (experts updated independently), (ii) *efficiency* (build once, query many), and (iii) *privacy* (real names never reach the agent). See companion illustration for the HyperRAG retrieval architecture at query-time.